Advanced Python Concepts

1. Decorators and Metaprogramming

Decorators provide a clean syntax for modifying or enhancing functions and classes without permanently altering their source code. Using the `@decorator_name` syntactic sugar above function definitions, developers can inject logging, access control, caching, and validation.

When writing decorators, it is best practice to use `functools.wraps` to preserve the original function's metadata (`__name__` and `__doc__`). Caching can be easily integrated using `functools.lru_cache`.

```
import functools def log_execution(func): @functools.wraps(func) def wrapper(*args, **kwargs): print(f'Executing {func.__name__}...') return func(*args, **kwargs) return wrapper
```

2. Generators, Iterators, and Itertools

Iterators implement the iterator protocol via `__iter__()` and `__next__()`. Generators simplify iterator creation using the `yield` keyword, pausing execution and maintaining state between calls without allocating memory for entire collections.

Generator expressions provide a memory-efficient shorthand similar to list comprehensions. Additionally, the `itertools` module offers powerful tools for efficient looping, such as `cycle`, `chain`, and `combinations`.

```
def fibonacci_gen(n): a, b = 0, 1 for _ in range(n): yield a a, b = b, a + b
```

3. Context Managers and the With Statement

Context managers handle resource allocation and cleanup reliably, ensuring that resources like file descriptors or database connections are safely released even if exceptions occur. They are invoked using the `with` statement.

Custom context managers can be built by implementing `__enter__` and `__exit__` magic methods, or more concisely by using the `@contextmanager` decorator from the `contextlib` standard module.

```
from contextlib import contextmanager @contextmanager def managed_resource(): print('Acquiring resource...') yield 'Resource' print('Cleaning up resource...')
```

4. Concurrency: Asyncio, Threading, and Multiprocessing

Python handles concurrency through three main paradigms depending on workload characteristics: I/O-bound vs CPU-bound tasks. Due to the Global Interpreter Lock (GIL), standard threads share a single CPU core and are ideal for I/O-bound tasks.

For heavy computational tasks, `multiprocessing` spawns independent processes bypassing the GIL. For cooperative multitasking, `asyncio` uses `async/await` syntax to manage thousands of concurrent network connections efficiently within a single event loop.

```
import asyncio async def fetch_data(): await asyncio.sleep(1) return {'data': 123}
```

5. Type Hints and Static Analysis

Introduced in PEP 484, type hints improve code maintainability and enable powerful static analysis tools like mypy. Python 3.10 introduced the union operator (`|`), and Python 3.12 introduced syntax for type aliases (`type Point = tuple[float, float]`).

```
from typing import Sequence, Optional def calculate_mean(numbers: Sequence[float]) -> Optional[float]: if not numbers: return None return sum(numbers) / len(numbers)
```